EE 495 Scientific Machine Learning— Final Project

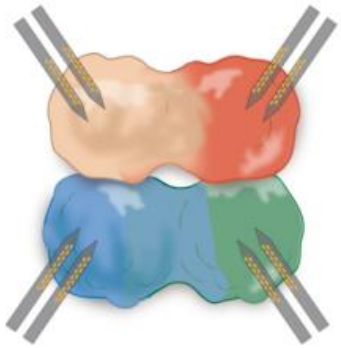
Scientific Machine Learning for Spike Detection and Classification in Neural Electrophysiology Data

Ziyue Wang

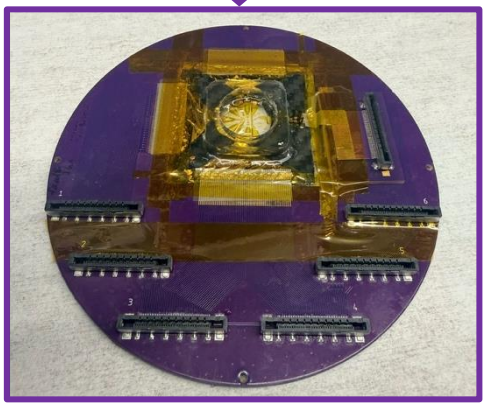
Northwestern University
Materials Science and Engineering

Motivation

Neural Signal Analysis in Organoid



Organoid system



Neural interface device

Neural interface devices combined with organoid systems enable recording of neural activity

These experiments generate large amounts of neural signal data that are:

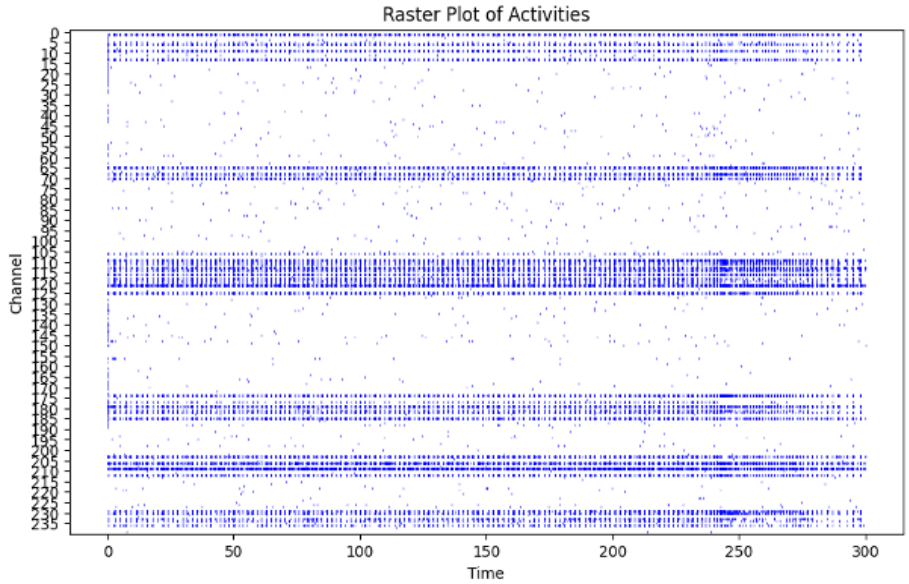
- Noisy time-series data
- High-dimensional
- Difficult to analyze

→ Accurate spike detection becomes challenging

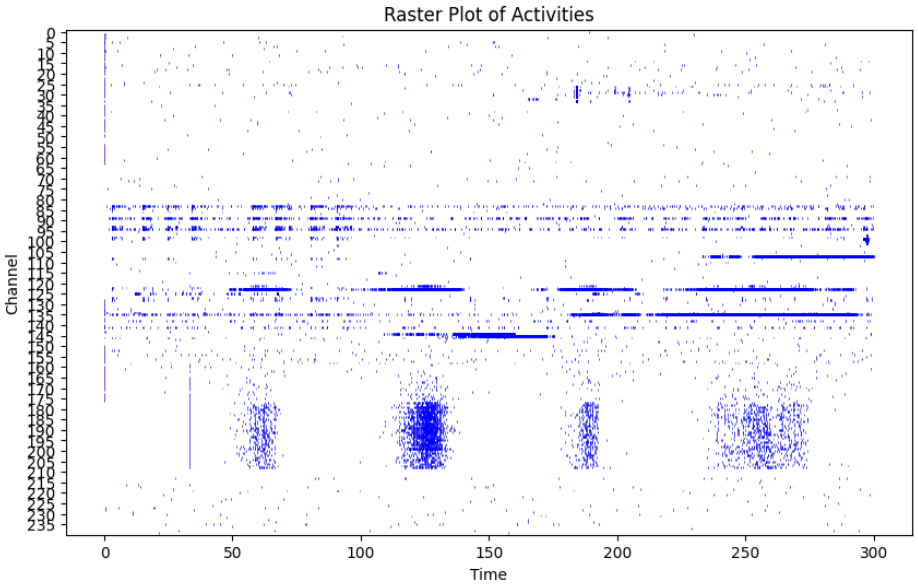
Traditional methods are sensitive to noise
→ Machine learning provides a more robust, data-driven solution

Problem

Spike Detection in Noisy Neural Signals



High SNR



Low SNR

Neural recordings contain both clean and noisy signals

Noise makes spike detection difficult

Threshold-based methods may fail in low SNR conditions

→ Need data-driven methods for reliable spike detection

Goal

Spike Detection Using Machine Learning

Build upon an existing spike sorting pipeline

Combine clustering methods with neural networks

Detect spikes in noisy electrophysiology data

Improve robustness compared to traditional methods

Raw signal
↓
Detection & clustering
↓
Neural network
↓
Spike labels

Spike Detection Pipeline

Traditional pipeline vs ML-based pipeline

Traditional pipeline

Raw signal
↓
Filtering (band-pass)
↓
Peak detection (threshold-based)
↓
UMAP (dimensionality reduction)
↓
DBSCAN (clustering)
↓
Manual selection (expert-based)
↓
Spike raster / analysis

ML-based pipeline

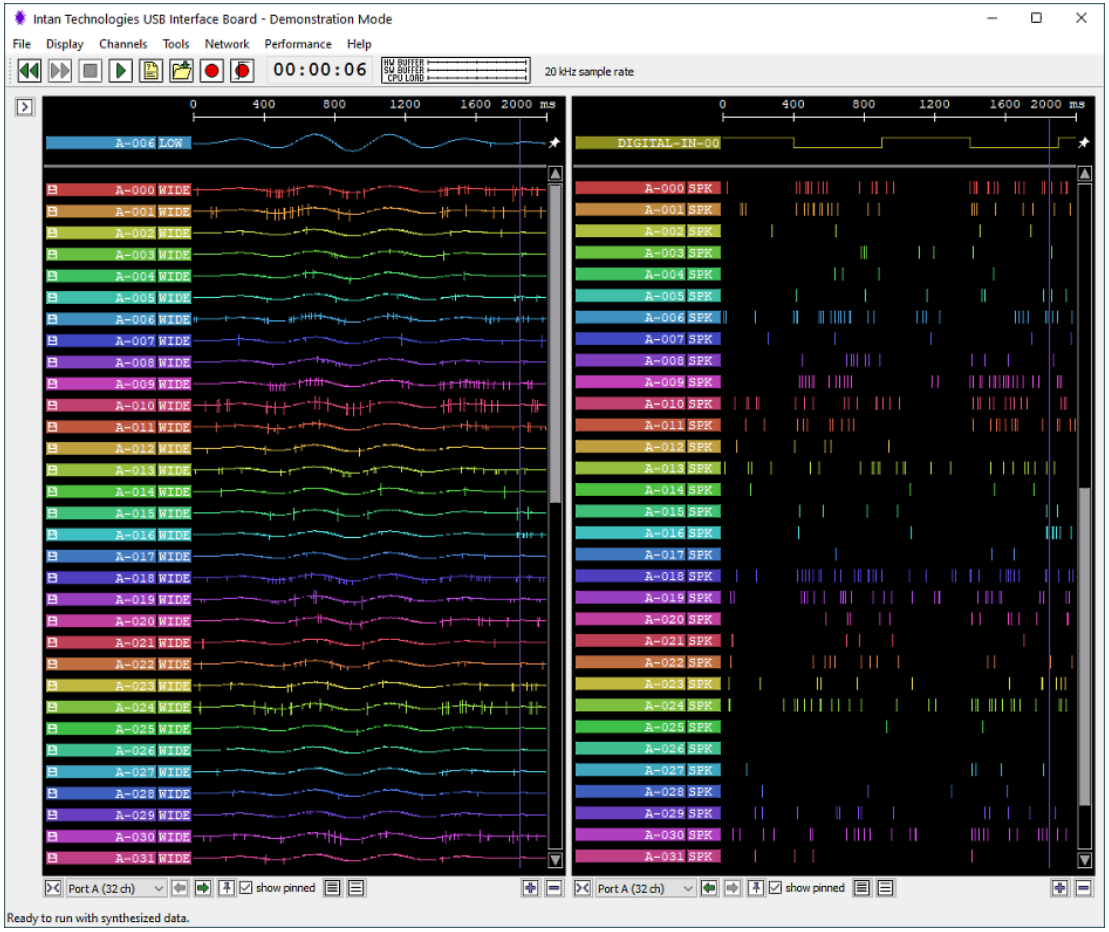
Raw signal
↓
Filtering (band-pass)
↓
Peak detection (threshold-based)
↓
UMAP (dimensionality reduction)
↓
DBSCAN (clustering)
↓
CNN (data-driven)
↓
Spike raster / analysis

Raw Signal

Northwestern

High-density neural recording setup

- 240-channel neural interface array
- Data acquired using the Intan system
- Sampling rate: 20 kHz
- Amplifier bandwidth: 0.1–3000 Hz
- 60 Hz notch filter
- Initial spike threshold: −24 µV
- Recording duration: ~5 min per experiment
- ~600 MB per minute (~3 GB per recording)



Data Preparation and Filtering

From raw signal to cleaned signal

Data preparation:

- Load Intan recording files (.rhd)
- Concatenate recordings into a continuous time series
- Select time window for analysis

→ Raw neural signals contain drift and noise

Filtering:

- Band-pass filtering (250–3000 Hz)
  → isolate spike frequency range
- Butterworth filter (order = 3)
- Zero-phase filtering (filtfilt)
  → avoid phase distortion

→ Enhance spike signals and improve SNR

Spike Detection

From cleaned signal to spike waveforms

Spike detection:

- Detection performed on band-pass filtered signals

- Adaptive threshold:
  $\theta = k \cdot \text{RMS (noise level)}$

- Detect both positive and negative peaks

- Minimum peak distance (peak_gap)
  → prevent double counting

- Amplitude limit → remove artifacts

- Extract spike waveforms (~3 ms window)
  → Output: spike waveforms for clustering and CNN

Dimensionality Reduction (UMAP)

From spike waveforms to feature space

UMAP for dimensionality reduction:

- Input: high-dimensional spike waveforms
- Reduce to low-dimensional embedding (2D)
- Preserve local similarity between spike shapes
- Similar spikes form clusters in embedding space

→ Output: low-dimensional features for clustering

Clustering (DBSCAN)

Identifying spike clusters in embedding space

DBSCAN for clustering:

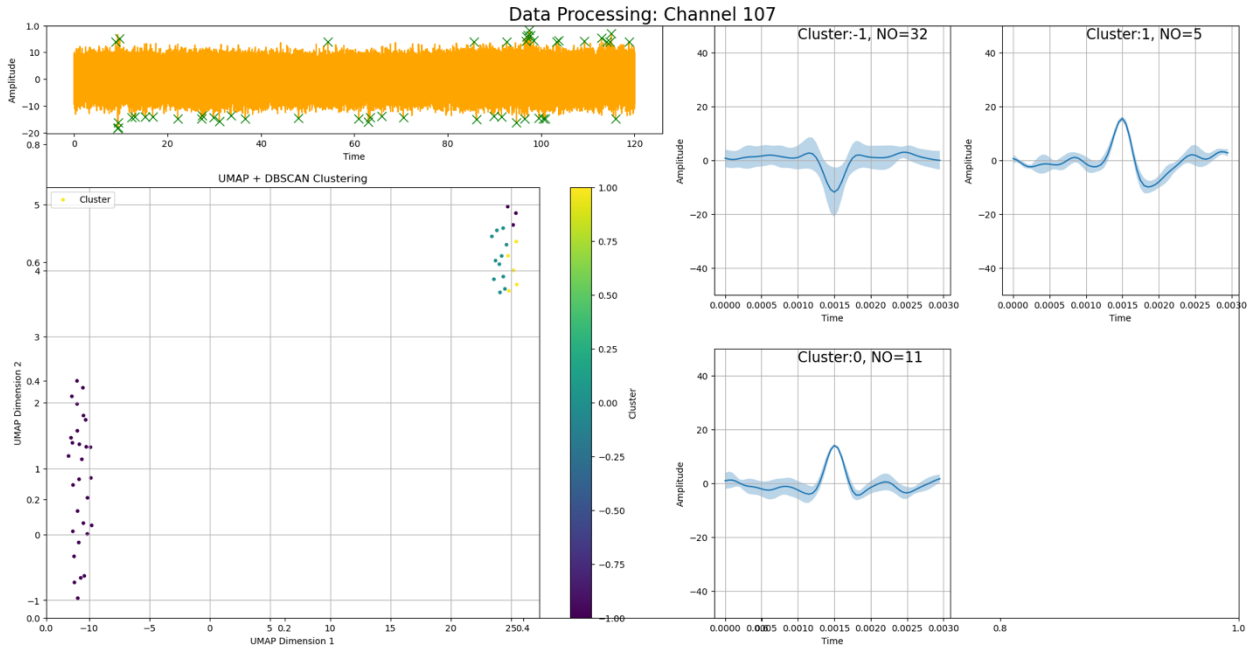
- Input: low-dimensional embedding from UMAP
- Density-based clustering method
- Groups nearby points into clusters
- Identifies noise and outliers (label = -1)
- No need to predefine number of clusters

→ Output: cluster labels for spike sorting

Limitations of Traditional Spike Sorting

Northwestern

Challenges in noisy neural recordings



- Sensitive to noise
- Overlapping spike shapes
- Unstable clustering
- Requires manual tuning

→ Motivation for ML-based approach

CNN-based Spike Classification

From clustering to supervised learning

Proposed approach:

- Use CNN to classify spike waveforms
- Input: detected spike segments
- Labels obtained from clustering (DBSCAN)
- Learn features automatically from data
- Replace manual clustering with learned model

→ More robust to noise and variability

CNN Architecture and Training

Northwestern

Lightweight 1D CNN for waveform classification

Model:

- 1D CNN for waveform classification
- Convolution layers extract temporal features
- Adaptive pooling for variable-length inputs
- Fully connected layer for classification

Training:

- Remove noise samples (label = -1)
- Train/test split
- Cross-entropy loss
- Incremental learning supported

→ Output: trained spike classifier

Results

CNN-based spike classification

Dataset:
- 23k spike waveforms extracted
- Noise removed using DBSCAN clustering

CNN training:
- 7 spike classes detected
- Incremental learning supported

Performance:
- Test accuracy ≈ 75%
- Stable convergence during training

```
epoch 0: loss=1.5209, test_acc=0.682
epoch 1: loss=0.6727, test_acc=0.684
epoch 2: loss=0.6318, test_acc=0.754
epoch 3: loss=0.6072, test_acc=0.754
epoch 4: loss=0.5916, test_acc=0.753
```

Conclusion:
CNN-based classification improves robustness compared to traditional spike sorting

Conclusion

- Developed a spike detection and classification pipeline for neural recordings
- Implemented traditional method:
  Filtering → Detection → UMAP → DBSCAN
- Proposed a CNN-based spike classification approach
- Improved robustness in noisy neural data
- Approach can be extended to large-scale neural interface experiments

Future Work

- Train with more recording data
- Improve CNN architecture
- Real-time spike classification
- Apply to larger neural arrays
- Integrate with neural interface experiments

Thank you